Main Project

Hardware for Artificial Intelligence and Machine Learning (HW4AI)

ECE 410/510
Spring 2026

Overview

The main project runs the full length of the term. You will design, implement, verify, and benchmark a custom co-processor chiplet that accelerates a part of an AI/ML algorithm of your choice. The chiplet must be described in a hardware description language (SystemVerilog preferred), synthesizable using OpenLane 2, and connected to a host system through a well-defined, standard hardware interface.
The project is individual. If multiple students work on the same algorithm and target kernel, they are competing, not collaborating. Shared code or designs constitute academic misconduct.
The project is assessed at four milestones across the term. Each milestone is graded *Satisfactory* / *Not Yet*. The final examination during finals week is based on your submitted M4 package. It will ask follow-up questions about your design decisions, tradeoffs, and the concepts you applied. You are expected to explain and defend every aspect of your design.

Project goals

Design goal: Produce a synthesizable HDL description of a co-processor chiplet that measurably accelerates a target AI/ML kernel relative to a software baseline.
Analysis goal: Apply the roofline model, arithmetic intensity analysis, and precision/energy tradeoff reasoning to justify every major architectural decision.
Interface goal: Connect the chiplet to a host system through a standard, well-specified hardware interface. The interface must be fully described in your documentation and verified in simulation.
Communication goal: Produce a design justification report that a knowledgeable reader could use to reproduce your architectural decisions, including decisions you tried that did not work.

Constraints and rules

You must

- Choose an AI/ML algorithm with at least one computationally dominant kernel that benefits from hardware acceleration.
- Establish a software baseline benchmark in Week 1 that you will use to measure speedup at M4.
- Apply the roofline model to characterize your target kernel before designing the accelerator. This analysis must appear in M1.
- Specify and implement a standard hardware interface between your chiplet and the host (see Interface section below).
- Describe your design in SystemVerilog (Verilog or VHDL is acceptable). The design must synthesize without errors using OpenLane 2.
- Write a testbench that verifies functional correctness for at least a small representative input.
- Document every major design decision with a justification grounded in data from your profiling and analysis, not in general claims about the technology.

You must not

- Rely on an existing GPU, CPU, or off-the-shelf accelerator as the accelerating element. A CPU may execute non-accelerated code only.
- Submit a design that passes simulation but has not been attempted through synthesis.
- Submit documentation generated by an LLM that you have not critically reviewed and revised to accurately reflect your actual design. The final examiner will probe this.
- Work with another student on design or implementation. Discussion of concepts is fine; sharing code or HDL is not.

Alternative project track
Instead of a co-processor chiplet, you may design a standalone chiplet that includes a soft processor core (e.g., a minimal RISC-V implementation). This track is appropriate if your accelerator is tightly coupled with custom control logic. The same interface, synthesis, and analysis requirements apply. Discuss this option with the instructor before Week 2.

Hardware interface requirement

Every chiplet must expose a well-defined, standard hardware interface to a host system. "Well-defined" means the interface is governed by a published specification, has a clear transaction protocol, and could in principle be connected to a real host without modification to the interface logic.
The interface serves two purposes: it makes your accelerator realistic as an engineering artifact, and it constrains the design in ways that expose real tradeoffs (latency, bandwidth, transaction overhead) that you must reason about analytically.

Interface	Complexity	Typical bandwidth	Notes
SPI	Low	Up to ~50 Mbit/s	Good for low-throughput edge inference. Simple to implement and verify. Appropriate if your kernel runs on an MCU-class host.
I²C	Low	Up to ~3.4 Mbit/s	Very low bandwidth; only suitable if your accelerator processes tiny models or sensor data. Requires strong justification.
AXI4-Lite	Medium	Varies by bus width	ARM AMBA standard; widely used in FPGA SoC designs. Good choice for control-plane traffic and register maps. Pair with AXI4 Stream for data.
AXI4 Stream	Medium	High (configurable)	Unidirectional streaming; natural fit for inference pipelines with continuous data flow. Pairs well with AXI4-Lite control.
PCIe (Gen3/4)	High	~1–16 GB/s per lane	Appropriate for data-center inference accelerators. Requires a PCIe endpoint controller IP; implementation complexity is significant. Suitable for 510-level students.
UCIe (chiplet)	High	Up to ~100 GB/s	Universal Chiplet Interconnect Express. Appropriate for multi-die designs. Primarily an architectural study at this scale.

You must select one interface and justify the choice in your M1 document. The justification must address: (1) the bandwidth requirement of your kernel at the operating point you are targeting, (2) whether the interface becomes the bottleneck (i.e., whether your design is interface-bound on the roofline), and (3) why this interface is appropriate for the host platform you are assuming.

Interface deliverables by milestone

- **M1:** Interface selection with bandwidth analysis. Show that the interface can sustain your target throughput, or explain and quantify the limitation if it cannot.
- **M2:** Interface module in HDL with a functional testbench. The testbench must exercise at least one complete write transaction and one complete read or response transaction.
- **M3:** Interface integrated with the accelerator core. Co-simulation must demonstrate data flowing from host through the interface into the compute engine and results returning to the host.
- **M4:** Interface included in synthesis results. Timing closure on the interface logic must be demonstrated or the failure documented and analyzed.

Dos and don'ts

Dos

- Pick something that genuinely interests you and that you want to understand at the hardware level.
- Pick something that will challenge you. Failure should be an option, and partial results with rigorous analysis are more valuable than a shallow success.

- Start with a blank slate. It is fine to draw inspiration from published architectures, but your design must be your own work.
- Use LLMs aggressively for code generation, debugging, and documentation drafts. The course expects this. But verify everything the LLM produces and be prepared to explain it.
- Engage early with the roofline model. The single most common cause of scope failure is designing an accelerator that is interface-bound or memory-bound in a regime the student did not anticipate.
- Talk to the instructor early if your scope feels wrong. The Week 7 scope assessment is a late backstop, not the first line of defense.

Don’ts

- Do not submit LLM-generated HDL you cannot explain. The final exam will ask you to justify specific lines of code.
- Do not wait until Week 7 to attempt synthesis. Tools always surface constraints you did not anticipate. Attempt synthesis no later than Week 5.
- Do not treat the interface as an afterthought. If your chiplet has no realistic path to talking to a host, it is not a chiplet.
- Do not over-scope. A fully synthesized, interface-connected, benchmarked accelerator for a single convolutional layer is more valuable than an ambitious design that never synthesizes.

Getting started: Week 1 checklist

Complete the following before the end of the Week 1 codefest. These form the raw material for your M1 submission.

- Answer the first three Heilmeier questions for your project idea (https://www.darpa.mil/about/heilmeier-catechism). If you cannot answer question 2 (what is done today and what are the limits?), your scoping is not ready.
- Find or write a software implementation of your target algorithm in Python. Library use (NumPy, PyTorch, etc.) is permitted, but document the platform (CPU or GPU, hardware spec) and batch size so the M4 comparison is reproducible. Establish a baseline benchmark: record execution time, throughput (e.g., samples/sec or FLOPs/sec), and memory usage. This number is your M4 comparison point.
- Profile the software implementation. Use cProfile, line_profiler, or py-spy to identify the computationally dominant function or loop. Compute a rough arithmetic intensity for that kernel (FLOPs per byte of memory traffic). Plot it on the roofline for your laptop’s CPU or GPU.
- Select a hardware interface from the table above. Write one paragraph justifying the choice based on your arithmetic intensity and throughput analysis.
- Draw a high-level block diagram showing: your host, the interface, the chiplet boundary, the compute engine inside the chiplet, and the on-chip memory.
- Create your GitHub repository and push your initial profiling code and Heilmeier answers before the end of Week 1. All subsequent deliverables must be committed there.
- If you are considering the alternative project track (standalone chiplet with soft processor core), discuss it with the instructor before the end of Week 1. Approval is required before Week 2.

Useful tools

Profiling and analysis

- cProfile, line_profiler, py-spy — Python execution profiling and hotspot identification.
- memory_profiler — memory usage patterns over time.
- torchinfo, fvcore — MACs, parameter count, and activation memory for PyTorch models.
- Nsight Compute (NVIDIA) — GPU kernel profiling including roofline placement.
- Doxygen, StaticFG, CodeQL — call graphs and data-flow analysis for understanding algorithm structure.

HDL and synthesis

- SystemVerilog / Verilog — primary HDL. Use SystemVerilog features (interfaces, assertions) where they simplify the design.
- OpenLane 2 — RTL-to-GDSII flow. (https://github.com/efabless/openlane2)
- cocotb — Python-based hardware co-simulation and testbench framework.
- Verilator — fast open-source Verilog/SystemVerilog simulator.

LLM-assisted design

- LLMs (Claude, ChatGPT, Gemini, etc.) — code generation, debugging, documentation drafts. All generated HDL must be reviewed and verified.
- For SystemVerilog specifically: always simulate LLM-generated code before trusting it. Common failure modes include incorrect sensitivity lists, non-synthesizable constructs, and timing assumption errors.

Suggested application domains

The list below is illustrative. Any AI/ML application with a computationally dense kernel is eligible. Talk to the instructor if you have something in mind not listed here.

High arithmetic intensity (compute-bound candidates)

- Dense matrix multiplication for transformer self-attention or feed-forward layers.
- Convolutional layers for image classification or object detection (YOLO-style).
- Recurrent neural network cells (LSTM, GRU) with fixed sequence lengths.
- Hyperdimensional computing encoding (large dot products over binary vectors).

Mixed or memory-bound (harder to accelerate; justify carefully)

- Sparse attention or sparse matrix-vector multiply.
- Embedding lookup for recommendation systems.
- Anomaly detection over streaming sensor data.
- Graph neural network message passing.

Edge and low-power targets (suitable for SPI or I²C interface)

- Binary or ternary neural network inference for keyword spotting or gesture recognition.
- Spiking neural network inference for event-driven sensor data.
- Tiny ML: MobileNet-style depthwise separable convolutions at INT4 or INT8.

Milestones and deadlines

All milestones are graded *Satisfactory* / *Not Yet*. Milestones 1–3 may be revised once within one week of receiving feedback, using a revision token if additional revisions are needed. Milestone 4 is final and cannot be revised. A late submission is graded *Not Yet* and consumes one revision token. Revision tokens cannot be used for Milestone 4.

Milestone	Due date	Required content
M1	Sun, Apr 12	Refined Heilmeier answers to questions 1, 2, and 3. Software baseline benchmark (execution time, throughput, memory). Profiling results identifying dominant kernel. Roofline plot with your kernel placed on it. Arithmetic intensity calculation. HW/SW partition proposal with rationale. Interface selection with bandwidth justification. High-level block diagram showing host, interface, chiplet boundary, compute engine, and on-chip memory.
M2	Sun, May 3	First working simulation: HDL description of the compute core with a testbench demonstrating correct output for a representative input. Interface module in HDL with testbench exercising at least one write and one read/response transaction. Precision choice documented (FP32 / FP16 / BF16 / INT8 / other) with quantization error analysis if applicable.

Milestone	Due date	Required content
M3	Sun, May 24	Synthesis attempt using OpenLane 2. If synthesis succeeds: timing report, area report, and identification of the critical path. Attempt to run power estimation and report whether it succeeded; M4 requires a power estimate. If synthesis fails or scope was adjusted: documented explanation of what failed, what was changed, and what the revised scope is. Interface integrated with compute core; co-simulation demonstrating end-to-end data flow.
M4	Sun, Jun 7	Complete deliverable package: (1) source code in GitHub repository with clear README; (2) synthesis results (timing, area, power estimate); (3) benchmark comparison showing accelerator throughput vs. software baseline; (4) roofline plot showing where the final design sits relative to the target hardware; (5) design justification report explaining every major architectural decision with supporting data. The final exam draws directly from this package.

Design justification report (M4)

The design justification report is not a tutorial on your application domain and not a literature review. It is an engineering account of the decisions you made and why you made them. A reader who already understands the course material should be able to read your report and understand why your design is the way it is, what you considered and rejected, and what the remaining limitations are.

Required sections

- **Problem and motivation.** What kernel are you accelerating, and why does it benefit from custom hardware? Cite your profiling data.
- **Roofline analysis.** What is the arithmetic intensity of your target kernel? Is your design compute-bound or memory-bound, and at what point does the bottleneck shift? How did this analysis shape your architecture?
- **Precision and data format.** What numerical format did you use and why? If you quantized, what was the quantization error and how did you verify it was acceptable?
- **Dataflow and architecture.** What dataflow pattern (weight-stationary, output-stationary, etc.) did you implement, and why is it appropriate for your kernel? Describe the compute engine, memory hierarchy, and data path.
- **Hardware interface.** Which interface did you implement, why, and what is the effective bandwidth at your target throughput? Is your design interface-bound? If so, quantify the impact.
- **Verification.** How did you verify functional correctness? What test cases did you use and what do they cover?
- **Synthesis results.** Report area, timing (critical path and slack), and power estimate. Identify the dominant contributor to each.
- **Benchmark results.** Compare your accelerator's throughput and/or energy to the software baseline from M1. Explain any gap between measured and theoretical performance.
- **What did not work.** Be specific. What did you attempt that failed, what did you learn from it, and what would you do differently?

LLMs may be used to improve clarity and structure of the report. Any AI-generated text must be critically revised to accurately reflect your actual design. The final exam will ask you to explain specific claims in the report; if you cannot, the claim should not be there.

Final examination

During finals week, you will complete a final examination based on your M4 deliverable package. The exam format will be announced at a later stage. The exam is graded *Pass with Distinction / Pass / No Pass*. Students receiving *No Pass* may not retake the exam and revision tokens do not apply to the final examination. The best preparation is to understand your own design thoroughly. A student who built the

accelerator but cannot explain why the dataflow is weight-stationary, or why SPI was chosen over AXI, will not pass.

How the project fits into your grade

The four milestones count as the “Project milestones” component of the grade bundle tables in the syllabus. The final examination counts as the “Final exam” component. See the syllabus for full grade bundle requirements.

Questions? Post to the #project channel on Slack or bring them to office hours. Please do not email me about project scope. Those conversations should happen in person during codefests or during office hours.